

ALGORITMI E STRUTTURE DATI

ANALISI DEGLI ALGORITMI

AULA VIRTUALE DEL 27-10-2026

Complessità temporale: caso peggiore, migliore e medio

Partiamo dall'analisi del tempo di esecuzione su istanze pessime, ottime ed istanze «tipiche», a parità di dimensione. Data un'istanza I di dimensione n, indicando con *tempo(I)*, il tempo di esecuzione dell'algoritmo su quella istanza.

ANALISI DEGLI ALGORITMI

Caso peggiore *(worst case)*: quante operazioni eseguiamo nel caso peggiore, ovvero per le istanze di ingresso che comportano più lavoro per algoritmo.

$$Tw(n) = \text{istanze } \max\{tempo(I)\}$$

Per istanze I di dimensione n

ANALISI DEGLI ALGORITMI

Caso migliore *(best case)*: numero minimo di operazioni che eseguiamo nel caso di istanze di ingresso particolarmente favorevole.

$$Tb(n)=istanze\ min\{tempo(I)\}$$

Per istanze I di dimensione n

ANALISI DEGLI ALGORITMI

Caso medio *(average case)*: qual è il tempo impiegato nel caso medio o, su istanze «tipiche» per quel particolare problema? Assumendo ogni istanza I di dimensione n abbia probabilità P{I} di verificarsi.

$$Ta(n)=\Sigma \{P\{i\} * tempo(I)\}$$

Per istanze I di dimensione n

ANALISI DEGLI ALGORITMI

Caso migliore *(best case)*: numero minimo di operazioni che eseguiamo nel caso di istanze di ingresso particolarmente favorevole.

$$Tb(n)=istanze\ min\{tempo(I)\}$$

Per istanze I di dimensione n

ANALISI DEGLI ALGORITMI

Osservazione: ma effettivamente l'analisi nel caso migliore fornisce informazioni utili? L'analisi nel caso medio per certi aspetti è sicuramente più significativa di quest'ultima.

In quest'ultimo caso, può risultare problematico definire esattamente la distribuzione di probabilità sulle istanze di ingresso. Dovendo fare delle ipotesi su tale distribuzione, l'accuratezza della nostra analisi dipenderà dalla validità delle ipotesi fatte.

Infine, potrebbe verificarsi il caso che l'algoritmo che stiamo analizzando nessuna istanza richieda il tempo «medio».

ANALISI DEGLI ALGORITMI

In virtù di quanto detto precedentemente, l'analisi nel caso peggiore è quella che si usa di solito, poiché fornisce garanzie sulle prestazioni dell'algoritmo nel caso più sfavorevole.

Inoltre, il tempo nel caso medio è sicuramente non superiore a quello nel caso peggiore, possiamo quindi utilizzare l'analisi nel caso peggiore per avere informazioni sui limiti superiori del caso medio, senza conoscere necessariamente la distribuzione dei dati di ingresso.

ANALISI DEGLI ALGORITMI

```
def ricercaSequenziale(lista, elemento):
    """
    Restituisce l'indice dell'elemento se trovato,
    altrimenti restituisce -1.
    """
    for i in range(len(lista)):
        if lista[i] == elemento:
            return i
    return -1


# Esempio di utilizzo
numeri = [4, 7, 12, 19, 23, 31]
da_cercare = 19

posizione = ricercaSequenziale(numeri, da_cercare)

if posizione != -1:
    print(f"Elemento trovato in posizione {posizione}")
else:
    print("Elemento non trovato")
```

ANALISI DEGLI ALGORITMI

Per questo algoritmo indichiamo con n=6 il numero di elementi nella lista dei numeri da ricercare ed analizziamo il tempo di esecuzione in funzione di n rispettivamente nel caso migliore, peggiore e medio.

Caso migliore: la ricerca sequenziale esegue un solo confronto, trovando immediatamente l'elemento x cercato (il numero 19) in prima posizione.

ANALISI DEGLI ALGORITMI

Caso peggiore: Il caso peggiore si ha quando l'elemento x cercato (il numero 19) è all'ultimo posto, oppure quando non è stato trovato nella lista dei numeri da ricercare. Pertando Tw(n)=n.

Caso medio: L'analisi del caso medio è più complessa. Sappiamo che il numero di confronti richiesti per trovare x è dato dalla posizione di x nella lista dei numeri, che indicheremo con pos(x).

ANALISI DEGLI ALGORITMI

```python
def ricercaSequenziale(lista, elemento):
    """
    Restituisce una tupla:
    (indice dell'elemento, numero di confronti eseguiti)
    Se non trovato, indice = -1.
    """
    confronti = 0

    for i in range(len(lista)):
        confronti += 1
        if lista[i] == elemento:
            return i, confronti

    return -1, confronti


# Esempio di utilizzo
numeri = [4, 7, 12, 23, 31, 19]
da_cercare = 19

posizione, num_confronti = ricercaSequenziale(numeri, da_cercare)

if posizione != -1:
    print(f"Elemento trovato in posizione {posizione}")
else:
    print("Elemento non trovato")

print(f"Numero di confronti eseguiti: {num_confronti}")
```

ANALISI DEGLI ALGORITMI

Facciamo un upgrade del precedente algoritmo di ricerca con l'implentazione e poi il confronto con un algoritmo di ricerca binaria.

Differenze rispetto alla ricerca sequenziale:

- La lista deve essere ordinata.
- Riduce drasticamente i confronti:
  - ricerca sequenziale → fino a n confronti
  - ricerca binaria → circa $\log_2(n)$ confronti

ANALISI DEGLI ALGORITMI

```
def ricercaBinariaIter(lista, elemento):
    """
    Restituisce una tupla:
    (indice dell'elemento, numero di confronti eseguiti)
    Se non trovato, indice = -1.

    ATTENZIONE: la lista deve essere ordinata!
    """
    sinistra = 0
    destra = len(lista) - 1
    confronti = 0

    while sinistra <= destra:
        medio = (sinistra + destra) // 2
        confronti += 1

        if lista[medio] == elemento:
            return medio, confronti
        elif lista[medio] < elemento:
            sinistra = medio + 1
        else:
            destra = medio - 1

    return -1, confronti
```

ANALISI DEGLI ALGORITMI

```
# Esempio di utilizzo (lista ORDINATA)
numeri = [4, 7, 12, 19, 23, 31]
da_cercare = 19

posizione, num_confronti = ricercaBinariaIter(numeri, da_cercare)

if posizione != -1:
    print(f"Elemento trovato in posizione {posizione}")
else:
    print("Elemento non trovato")

print(f"Numero di confronti eseguiti: {num_confronti}")
```

ANALISI DEGLI ALGORITMI

Caso migliore: si ha quando x=[L(n+1)/2], e pertanto Tb(n)=O(1).

Caso peggiore: si ha quando Tw(n)=O(log n).

Caso medio: si ha quando

$$\mathbf{Ta(n)} = \sum_{p=1}^{n} \frac{1}{n} * (\boldsymbol{numero\ confronti\ se\ x\ sta\ in\ posizione\ p})$$

ANALISI DEGLI ALGORITMI

```
def ricercaBinariaRic(lista, elemento, sinistra, destra, confronti=0):
    """
    Restituisce una tupla:
    (indice dell'elemento, numero di confronti eseguiti)
    Se non trovato, indice = -1.

    ATTENZIONE: la lista deve essere ordinata!
    """
    if sinistra > destra:
        return -1, confronti

    medio = (sinistra + destra) // 2
    confronti += 1

    if lista[medio] == elemento:
        return medio, confronti
    elif lista[medio] < elemento:
        return ricercaBinariaRic(lista, elemento, medio + 1, destra, confronti)
    else:
        return ricercaBinariaRic(lista, elemento, sinistra, medio - 1, confronti)
```

ANALISI DEGLI ALGORITMI

```
# Esempio di utilizzo (lista ORDINATA)
numeri = [4, 7, 12, 19, 23, 31]
da_cercare = 19

posizione, num_confronti = ricercaBinariaRic(numeri, da_cercare, 0, len(numeri) - 1)

if posizione != -1:
    print(f"Elemento trovato in posizione {posizione}")
else:
    print("Elemento non trovato")

print(f"Numero di confronti eseguiti: {num_confronti}")
```

ANALISI DEGLI ALGORITMI

Cosa cambia rispetto alla versione iterativa:

- Usa la ricorsione invece del ciclo while.
- I parametri sinistra e destra vengono aggiornati a ogni chiamata.
- Il contatore confronti viene passato tra le chiamate per tracciare il numero totale.

ANALISI DEGLI ALGORITMI

Il tempo di esecuzione di algoritmi ricorsivi può essere espresso tramite relazioni di ricorrenza: il tempo richiesto da una procedura è uguale al tempo speso all'interno della procedura più il tempo speso per le chiamate ricorsive. Ad esempio, la relazione di ricorrenza relativamente all'algoritmo di ricerca binaria è $T(n) = O(1) + T(n/2)$.

Nel caso peggiore:

$$T(n) = 2 + T\left(\left[\frac{n-1}{2}\right]\right)$$

ANALISI DEGLI ALGORITMI

La notazione asintotica è uno strumento fondamentale in quanto ci serve per descrivere come cresce il tempo (o spazio) di un algoritmo al crescere della dimensione dell'input n, ignorando costanti e dettagli meno rilevanti.
Perché utilizzare la notazione asintotica? In primis per semplificare le formule, ignorando i fattori costanti ed altri dettagli ininfluenti.

ANALISI DEGLI ALGORITMI

Inoltre, cosa più importante la notazione asintotica ci consentirà di confrontare due algoritmi in modo semplice (confrontabilità).
Le notazioni che useremo per descrivere il tempo di esecuzione asintotico di un algoritmo sono definite in termini di funzioni il cui dominio è normalmente l'insieme N dei numeri naturali oppure R dei numeri reali.

ANALISI DEGLI ALGORITMI

O grande (Big-O) o limite superiore
Indica il caso peggiore: quanto al massimo può crescere il costo di un algoritmo.
In altre parole, ci dice che una funzione non cresce più velocemente di un certo tasso di crescita, determinato dal termine di ordine superiore.

Ad esempio, data la funzione $7n^3 + 100n^2 - 20n + 6$
Il termine di ordine superiore è $7n^3$ quindi diremo che il tasso di crescita di questa funzione è n^3.
Poiché questa funzione non cresce più velocemente di n^3 diremo che O(n^3).

ANALISI DEGLI ALGORITMI

Θ (Theta) o limite stretto
Indica il comportamento esatto (sia superiore che inferiore): descrive la crescita "reale" dell'algoritmo.

"L'algoritmo cresce come T(n) , né più né meno (a fattori costanti)."

Esempi:

- Somma di un array → Θ(n)
- Accesso a un elemento in array → Θ(1)

ANALISI DEGLI ALGORITMI

Ω (Omega) o limite inferiore
Indica il caso migliore: il minimo tempo necessario.

“L’algoritmo crescerà almeno come T(n).”

Esempi:

- Ricerca sequenziale → Ω(1) (se trovi subito l’elemento)
- Ricerca binaria → Ω(1)

ANALISI DEGLI ALGORITMI

Analisi ammortizzata

In molte situazioni pratiche, ci troviamo ad eseguire un algoritmo ripetutamente nel tempo. Si pensi alle ricerche effettuate da un motore di ricerca sul web.
Sostanzialmente vengo effettuate queries su una base dati per la ricerca, aggiornamento, inserimento, cancellazione.

L'analisi ammortizzata studia le prestazioni medie di una sequenza di operazioni su una collezione di dati.

ANALISI DEGLI ALGORITMI

Analisi ammortizzata

Rispetto a quanto visto finora, è che nell'analisi ammortizzata le probabilità non entrano in gioco in nessun modo.

Per caratterizzare le prestazioni di un algoritmo in media su una sequenza di operazioni, parleremo di costo ammortizzato.

$T_n(n) = \frac{T(n,k)}{k}$, dove T(n,k) è il tempo totale richiesto dall'algoritmo nel caso peggiore per tutte le k operazioni su istanze di dimensione n.

ANALISI DEGLI ALGORITMI

Referenze

- Algoritmi e Strutture Dati – Demetrescu, Finocchi, Italiano
- Introduzione agli algoritmi e strutture dati – Cormen, Leiserson, Rivest, Stein

Risorse Python

https://runestone.academy/ns/books/published/pythonds/index.html?utm_source=chatgpt.com

https://github.com/0bprashanthc/algorithm-books/blob/master/Data%20Structures%20%26%20Algorithms%20in%20Python.pdf?utm_source=chatgpt.com

https://edu.anarcho-copy.org/Programming%20Languages/Python/Python%20Data%20Structures%20and%20Algorithms.pdf?utm_source=chatgpt.com

https://do1.dr-chuck.com/pythonlearn/IT_it/pythonlearn.pdf?utm_source=chatgpt.com

